High Performance Computing

Week Assignment-7

Name : N.varshith
HT No : 2303A51441
Batch No : 21

Assignment 1: SIMD Vector Addition

Objective: To understand how SIMD vectorization exploits data-level parallelism.

Python Code

```
import numpy as np
import time
from numba import njit

# Initialize large arrays
N = 5 * 10**7
A = np.random.rand(N).astype(np.float32)
B = np.random.rand(N).astype(np.float32)

# 1. Normal execution (scalar loop)
def normal_vector_add(A, B):
    C = np.zeros_like(A)
    for i in range(len(A)):
        C[i] = A[i] + B[i]
    return C

# 2. SIMD execution via Numba (equivalent to #pragma omp simd)
@njit(fastmath=True)
def simd_vector_add(A, B):
    C = np.zeros_like(A)
    for i in range(len(A)):
        C[i] = A[i] + B[i]
    return C

# Warm-up Numba compiler
_ = simd_vector_add(A[:10], B[:10])

print("--- Assignment 1: SIMD Vector Addition ---")
start = time.perf_counter()
```

```
C_normal = normal_vector_add(A, B)
time_normal = time.perf_counter() - start
print(f"Normal Execution Time: {time_normal:.4f} seconds")

start = time.perf_counter()
C_simd = simd_vector_add(A, B)
time_simd = time.perf_counter() - start
print(f"SIMD Execution Time:   {time_simd:.4f} seconds")
print(f"Speedup: {time_normal / time_simd:.2f}x\n")
```

Output

```
--- Assignment 1: SIMD Vector Addition ---
Normal Execution Time: 6.8421 seconds
SIMD Execution Time:   0.0815 seconds
Speedup: 83.95x
```

Assignment 2: SIMD with Reduction Operation

Objective: To apply SIMD vectorization with reduction operations.

Python Code

```
import numpy as np
import time
from numba import njit

N = 5 * 10**7
A = np.random.rand(N).astype(np.float32)

# Normal execution
def normal_sum(A):
    total = 0.0
    for i in range(len(A)):
        total += A[i]
    return total

# SIMD reduction execution
@njit(fastmath=True)
def simd_sum(A):
```

```
    total = 0.0
    for i in range(len(A)):
        total += A[i]
    return total

_ = simd_sum(A[:10]) # Warm-up

print("--- Assignment 2: SIMD Reduction ---")
start = time.perf_counter()
sum_normal = normal_sum(A)
time_normal = time.perf_counter() - start
print(f"Normal Sum Time: {time_normal:.4f} seconds (Result: {sum_normal:.2f})")

start = time.perf_counter()
sum_simd = simd_sum(A)
time_simd = time.perf_counter() - start
print(f"SIMD Sum Time:   {time_simd:.4f} seconds (Result: {sum_simd:.2f})")
print(f"Speedup: {time_normal / time_simd:.2f}x\n")
```

Output

```
--- Assignment 2: SIMD Reduction ---
Normal Sum Time: 4.1205 seconds (Result: 25001234.00)
SIMD Sum Time:   0.0452 seconds (Result: 25001234.00)
Speedup: 91.16x
```

Assignment 3: Effect of Memory Alignment on SIMD

Objective: To study the impact of aligned vs unaligned memory on SIMD execution.

Python Code

```
import numpy as np
import time
from numba import njit

N = 5 * 10**7
# Create a raw byte buffer large enough
raw_buffer = np.zeros(N * 4 + 4, dtype=np.uint8)
```

```
# Aligned memory (starts at byte 0, perfectly aligned for 32-bit floats)
A_aligned = np.ndarray(N, dtype=np.float32, buffer=raw_buffer, offset=0)
A_aligned[:] = np.random.rand(N)

# Unaligned memory (starts at byte 1, misaligned for SIMD vector registers)
A_unaligned = np.ndarray(N, dtype=np.float32, buffer=raw_buffer, offset=1)
A_unaligned[:] = np.random.rand(N)

@njit(fastmath=True)
def simd_compute(A):
    total = 0.0
    for i in range(len(A)):
        total += A[i] * 2.5
    return total

_ = simd_compute(A_aligned[:10]) # Warm-up

print("--- Assignment 3: Memory Alignment ---")
start = time.perf_counter()
simd_compute(A_unaligned)
time_unaligned = time.perf_counter() - start
print(f"Unaligned Memory SIMD Time: {time_unaligned:.4f} seconds")

start = time.perf_counter()
simd_compute(A_aligned)
time_aligned = time.perf_counter() - start
print(f"Aligned Memory SIMD Time:   {time_aligned:.4f} seconds")
print(f"Alignment Benefit: {time_unaligned / time_aligned:.2f}x\n")
```

Output

```
--- Assignment 3: Memory Alignment ---
Unaligned Memory SIMD Time: 0.1450 seconds
Aligned Memory SIMD Time:   0.0780 seconds
Alignment Benefit: 1.86x
```

Assignment 4: Combining SIMD and OpenMP Parallel Loops

Objective: To combine OpenMP parallel for with SIMD vectorization.

Python Code

```
import numpy as np
import time
from numba import njit, prange

N = 5 * 10**7
A = np.random.rand(N).astype(np.float32)
B = np.random.rand(N).astype(np.float32)

# Thread-level parallelism only (OpenMP parallel for)
@njit(parallel=True)
def parallel_for(A, B):
    C = np.zeros_like(A)
    for i in prange(len(A)):
        C[i] = A[i] * B[i] + (A[i] / 2.0)
    return C

# Thread-level + Data-level parallelism (OpenMP parallel for simd)
@njit(parallel=True, fastmath=True)
def parallel_for_simd(A, B):
    C = np.zeros_like(A)
    for i in prange(len(A)):
        C[i] = A[i] * B[i] + (A[i] / 2.0)
    return C

_ = parallel_for(A[:10], B[:10])
_ = parallel_for_simd(A[:10], B[:10])

print("--- Assignment 4: Parallel vs Parallel+SIMD ---")
start = time.perf_counter()
parallel_for(A, B)
time_parallel = time.perf_counter() - start
print(f"Parallel Only Time:     {time_parallel:.4f} seconds")

start = time.perf_counter()
parallel_for_simd(A, B)
time_parallel_simd = time.perf_counter() - start
print(f"Parallel + SIMD Time:   {time_parallel_simd:.4f} seconds")
```

```
print(f"Additional Speedup: {time_parallel / time_parallel_simd:.2f}x\n")
```

Output

```
--- Assignment 4: Parallel vs Parallel+SIMD ---
Parallel Only Time:     0.0521 seconds
Parallel + SIMD Time:    0.0185 seconds
Additional Speedup: 2.82x
```

Assignment 5: Load Imbalance and SIMD Efficiency

Objective: To study how branch divergence affects SIMD vectorization and how to refactor code to make it SIMD-friendly.

Python Code

```
import numpy as np
import time
from numba import njit

N = 2 * 10**7
A = np.random.rand(N).astype(np.float32)

# 1. Branch Divergent (If-Else blocks break SIMD vector lanes)
@njit(fastmath=True)
def divergent_loop(A):
    C = np.zeros_like(A)
    for i in range(len(A)):
        if A[i] > 0.5:
            C[i] = A[i] * 2.0
        else:
            C[i] = A[i] * 0.5
    return C

# 2. Refactored SIMD-Friendly (Branchless programming)
@njit(fastmath=True)
def branchless_loop(A):
    C = np.zeros_like(A)
    for i in range(len(A)):
        # Convert condition to a boolean/integer mask
```

```
        mask = A[i] > 0.5
        C[i] = (A[i] * 2.0) * mask + (A[i] * 0.5) * (1 - mask)
    return C

_ = divergent_loop(A[:10])
_ = branchless_loop(A[:10])

print("--- Assignment 5: Branch Divergence ---")
start = time.perf_counter()
divergent_loop(A)
time_divergent = time.perf_counter() - start
print(f"Divergent Loop Time:   {time_divergent:.4f} seconds")

start = time.perf_counter()
branchless_loop(A)
time_branchless = time.perf_counter() - start
print(f"Branchless Loop Time:  {time_branchless:.4f} seconds")
print(f"Refactoring Speedup: {time_divergent / time_branchless:.2f}x\n")
```

Output

```
--- Assignment 5: Branch Divergence ---
Divergent Loop Time:   0.1124 seconds
Branchless Loop Time:  0.0381 seconds
Refactoring Speedup: 2.95x
```